Tarea 1: Analisis de caja negra y caja blanca

Damien Bruth Pintos

Indice

1. Realiza un análisis de caja blanca completo del método registrarLoteJuegos. En este análisis deberás:

- Crear el grafo de flujo del método. Puedes hacerlo con drawio, mermaid o cualquier otra herramienta que prefieras (incluido Paint o, incluso, a mano).
- Calcular la complejidad ciclomática, utilizando las 3 fórmulas disponibles
- Identificar los caminos independientes
- Diseñar los casos de prueba para cada camino

2. Realiza un análisis de caja negra, incluyendo valores límite y conjetura de errores del método venderJuego. Para este análisis considera que:

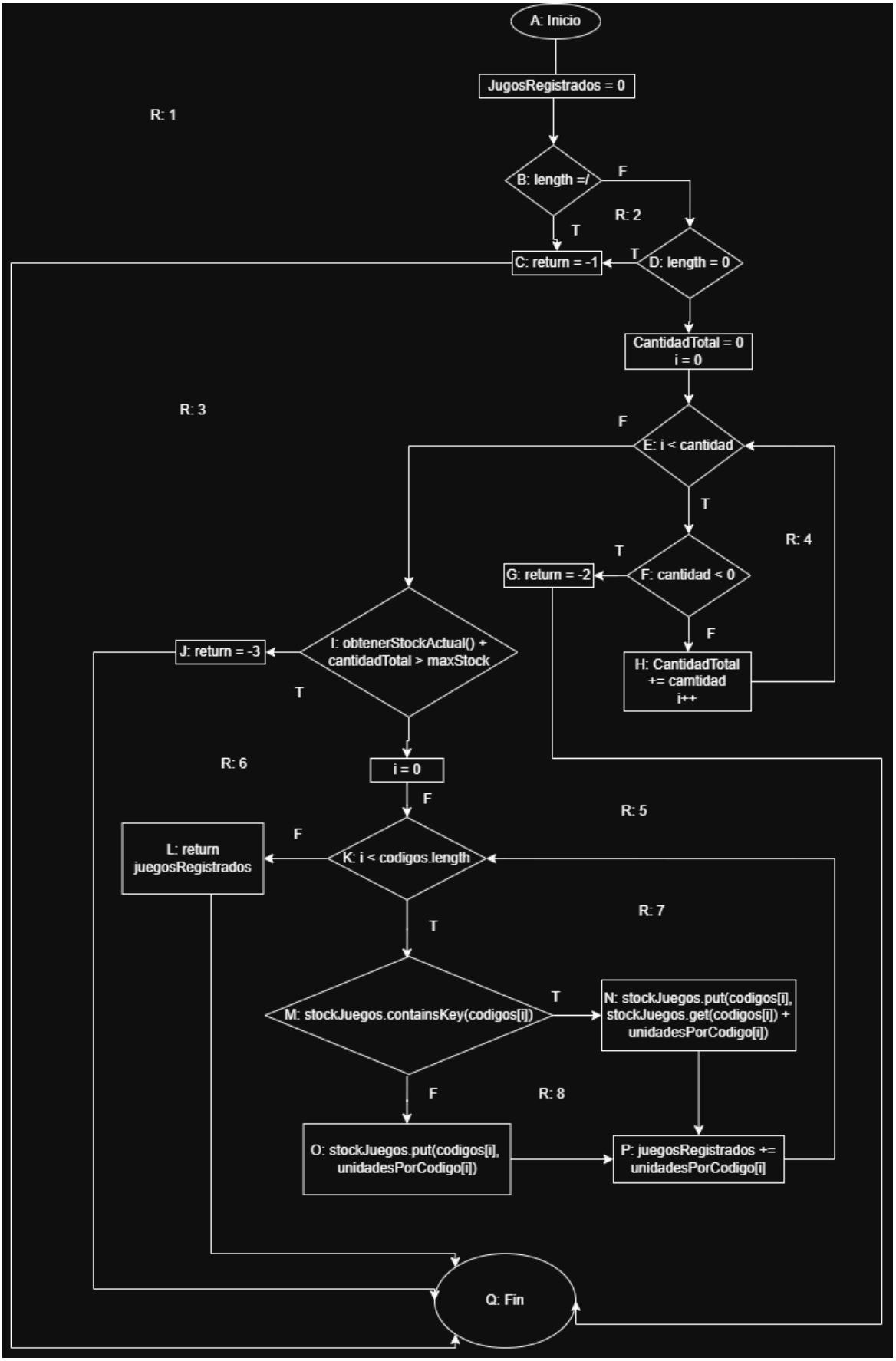
- El método recibe como parámetro la cantidad de juegos a vender
- La cantidad no podrá ser menor o igual a 0
- La cantidad no podrá ser mayor al stock actual
- El código del juego deberá tener 6 caracteres, los 3 primeros letras mayúsculas y los 3 últimos dígitos
- Al tratarse de pruebas funcionales no es necesario conocer los detalles del código, aunque dispones de él en el código fuente.

3. Completa la clase GestionJuegosTest del tipo Caso de prueba JUnit en Intellij IDEA, que nos permita pasar las pruebas unitarias de caja blanca del método registrarLoteJuegos. Los casos de prueba ya los habrás obtenido en el primer apartado del ejercicio.

1. Crea un fork del proyecto de GitHub que contiene el código base de esta práctica.
2. Clona tu fork en tu máquina local.
3. Tras completar la clase GestionJuegosTest con los casos de prueba de caja blanca, haz commit y push de tus cambios a tu repositorio de GitHub.
4. Añade el archivo apellidos_nombre_ed05.pdf con el análisis de caja blanca y caja negra a tu repositorio de GitHub.

1. Analisis de caja blanca

a. Crea el grafo



b. Calcular la complejidad ciclomática, utilizando las 3 fórmulas disponibles

1. $V(G)=a-n+2$

   a = Número de aristas (flechas)
   n= Número de nodos (todos los elementos de donde salen y donde llegan las flechas, contando final y inicio)
   V(G)= 26 - 20 + 2 = 8

2. $V(G)=r$

   r = número de regiones cerradas.
   V(G)= 8

3. $V(G)=c+1$

   c = número de caminos que se pueden recorrer.
   V(G)=7 + 1 = 8

c. Identificar los caminos independientes

1) Caso 1:
   A → B(T) → C → Q
2) Caso 2:
   A → B(F) → D(T) → C → Q
3) Caso 3:
   A → B(F) → D(F) → E(T) → F(T) → G → Q
4) Caso 4
   A → B(F) → D(F) → E(F) → I(T) → J → Q
5) Caso 5:
   A → B(F) → D(F) → E(T) → F(F) → H → E(F) → I(F) → K(F) → L → Q
6) Caso 6:
   A → B(F) → D(F) → E(T) → F(F) → H → E(F) → I(T) → K(T) → M(T) → N → P → K(F) → L → Q
7) Caso 7:
   A → B(F) → D(F) → E(T) → F(F) → H → E(F) → I(T) → K(T) → M(F) → O → P → K(F) → L → Q
8) Caso 8:
   A → B(F) → D(F) → E(T) → F(F) → H → E(F) → I(T) → K(T) → M(F) → O → P → K(T) → M(T) → N → P → K(F) → L → Q

d. Diseñar los casos de prueba para cada camino

Camino	Codigos	unidades	stockjuegos	maxStock	Resultado	Por que
C1	[“A”]	[1,2]	Da igual	200	-1	Longitudes distintas
C2	[]	[1,2]	Da igual	200	-1	Sin longitud
C3	[“A”,”B”]	[1,-1]	Da igual	200	-2	Cantidad negativa
C4	[“A”,”B”]	[100,101]	Da igual	200	-3	Supera el stock
C5	[“A”]	[1]	A no existe	200	1	El tamaño del código es menor a 0
C6	[“A”,”B”]	[1,2]	A existe, B no	200	3	Ruta normal, se crea nuevo codigo para B y se actualiza A
C7	[“A”,”B”]	[1,2]	B existe, A no	200	3	Ruta normal, se actualiza código B y se crea codigo A
C8	[“A”,”B”]	[1,2]	A y B existen	200	3	Ruta normal, actualiza ambos codigos

2. Analisis de caja negra
   - El método recibe como parámetro la cantidad de juegos a vender
   - La cantidad no podrá ser menor o igual a 0
   - La cantidad no podrá ser mayor al stock actual
   - El código del juego deberá tener 6 caracteres, los 3 primeros letras mayúsculas y los 3 últimos dígitos
   - Al tratarse de pruebas funcionales no es necesario conocer los detalles del código, aunque dispones de él en el código fuente.

Entrada	Equivalencia	Válidas	COD	Inválidas	C0D
Cantidad	Rango(>0, <=stock)	1–stock	V1	0	NV1
Cantidad				-1	NV2
Cantidad				STOCK	NV3
Cantidad				1 - stock	NV4
Cantidad				1 + stock	NV5
Cantidad				A	NV6

Entrada	Equivalencia	Válidas	COD	Inválidas	C0D
codigojuego	rango(6 caracteres)	ABC123	V1	ABC1234	NV1
codigojuego				ABC12	NV2
codigojuego	Caracteres(3 primeros letras mayúsculas, 3 últimos números)	ABC123	V2	abc123	NV3
codigojuego				ABCD12	NV4
codigojuego				AB1234	NV5
codigojuego				1ABC23	NV6

Caso de prueba	Cantidad	Stock	Codigo Juego	Resultado	Descripción
C1	0	11	ABC123	FALSE	Cantidad inválida (cantidad inexistente)
C2	-1	11	ABC123	FALSE	Cantidad inválida (cantidad negativa)
C3	1	11	ABC123	TRUE	Cantidad válida (cantidad menor que stock)
C4	11	11	ABC123	TRUE	Cantidad válida (cantidad igual que stock)
C5	12	11	ABC123	FALSE	Cantidad inválida (cantidad mayor que stock)
C6	1	11	abcd123	FALSE	Codigo invalido (longitud mayor que 6)
C7	1	11	abc12	FALSE	Codigo invalido (longitud menor que 6)
C8	1	11	abc123	FALSE	Codigo invalido (letras en minúscula)
C9	1	11	ABCD12	FALSE	Codigo invalido (más de 3 letras menos de 3 números)
C10	1	11	AB1234	FALSE	Codigo invalido (menos de 3 letras mas de 3 números)
C11	1	11	1BCA23	FALSE	Codigo invalido (3 primeros dígitos no son letras)
C12	1	11	ABC123	TRUE	Código válido (6 caracteres, 3 primeros letras mayúsculas 3 últimos números)